13th complex and string datatype

complex datatype

In [1]:
```
complex(10)
```

Out[1]:
```
(10+0j)
```

In [2]:
```
complex(10, 20, 30)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[2], line 1
----> 1 complex(10, 20, 30)

TypeError: complex() takes at most 2 arguments (3 given)
```

In [3]:
```
complex(10,20)
```

Out[3]:
```
(10+20j)
```

In [4]:
```
complex(2.3)
```

Out[4]:
```
(2.3+0j)
```

In [5]:
```
complex(2.3,4.5)
```

Out[5]:
```
(2.3+4.5j)
```

In [6]:
```
complex(23, 2.3)
```

Out[6]:
```
(23+2.3j)
```

In [7]:
```
complex(True)
```

Out[7]:
```
(1+0j)
```

In [8]:
```
complex(False)
```

Out[8]:
```
0j
```

In [1]:
```
complex(True,False)
```

Out[1]:
```
(1+0j)
```

In [2]:
```
complex(1)
```

Out[2]: (1+0j)

In [3]:
```
complex('10')
```

Out[3]: (10+0j)

In [4]:
```
complex('10','20')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[4], line 1
----> 1 complex('10','20')

TypeError: complex() can't take second arg if first is a string
```

In [5]:
```
complex(10, '20')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[5], line 1
----> 1 complex(10, '20')

TypeError: complex() second arg can't be a string
```

In [12]:
```
complex('ten')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[12], line 1
----> 1 complex('ten')

ValueError: complex() arg is a malformed string
```

string datatype

In [7]:
```
str(100)
```

Out[7]: '100'

In [8]:
```
str(2.3)
```

Out[8]: '2.3'

In [9]:
```
str(True)
```

Out[9]: 'True'

In [10]:
```
str(1+2j)
```

Out[10]: '(1+2j)'

In [11]:
```
str(2,3)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[11], line 1
----> 1 str(2,3)

TypeError: str() argument 'encoding' must be str, not int
```

python operators

Arithmetic operators---> +,-,*,%,/,//,**

In [13]:
```
x1, y1 = 10, 5
```

In [14]:
```
x1+y1
```

Out[14]: 15

In [15]:
```
x1-y1
```

Out[15]: 5

In [16]:
```
x1*y1
```

Out[16]: 50

In [17]:
```
x1/y1
```
Out[17]: 2.0

In [18]:
```
x1//y1
```
Out[18]: 2

In [19]:
```
x1**3
```
Out[19]: 1000

In [20]:
```
2**3
```
Out[20]: 8

In [22]:
```
x1 % y1  #gives reminder
```
Out[22]: 0

In [23]:
```
2 * 3
```
Out[23]: 6

In [24]:
```
2**5
```
Out[24]: 32

Assignment operator--- =, +=, -=, *=, /=

In [11]:
```
x=2
```

In [13]:
```
x = x + 2# if you want to increase by 2
```

In [12]:
```
x=x+2
x
```
Out[12]: 4

In [53]:
```
x+=2
x
```
Out[53]: 6

In [54]:
```
x*=2
x
```
Out[54]: 12

In [55]:
```
x*=2
x
```

Out[55]: 24

In [56]:
```
x * = 2
x
```

```
  Cell In[56], line 1
    x * = 2
        ^
SyntaxError: invalid syntax
```

In [57]:
```
x*=2
x
```

Out[57]: 48

In [58]:
```
x//=2
x
```

Out[58]: 24

In [14]:
```
a,b =5,6 # you can assign variables in one line as well
print(a)
print(b)
```

```
5
6
```

In [15]:
```
a=5
b=6
print(a)
print(b)
```

```
5
6
```

In [16]:
```
a
```

Out[16]: 5

In [17]:
```
b
```

Out[17]: 6

unary operators

unary means 1,binary means 2

In [66]:
```
#here we are applying unary minus(-) operator on the operand n,the value of n becom
```

In [67]:
```
n = 4 #negottion
n
```

Out[67]: 4

In [68]:
```
m = -n
m
```

Out[68]: -4

In [69]:
```
-n
```

Out[69]: -4

Relational operators

we use this operator for comparing

In [62]:
```
a=5
b=6
```

In [63]:
```
a<b
```

Out[63]: True

In [64]:
```
a>b
```

Out[64]: False

In [22]:
```
# a=b # we cannot use= operator that means it is assingning
```

In [23]:
```
a=b
```

In [67]:
```
a==b
```

Out[67]: True

In [25]:
```
a!=b
```

Out[25]: True

In [36]:
```
# here if i change b=6
b=6
```

In [37]:
```
a==b
```

Out[37]: True

In [38]:
```
a
```

Out[38]: 6

In [39]:
```
b
```

Out[39]: 6

In [40]:
```
#
```

In [41]:
```
a>b
```

Out[41]: False

In [42]:
```
a>=b
```

Out[42]: True

In [43]:
```
a<=b
```

Out[43]: True

In [44]:
```
a<b
```

Out[44]: False

In [45]:
```
a>b
```

Out[45]: False

In [46]:
```
b=7
```

In [47]:
```
a!=b
```

Out[47]: True

In [48]:
```
c=5
d=7
```

In [69]:
```
c==d
```

Out[69]: False

In [70]:
```
c=d
```

In [71]:
```
print(c)
print(d)
```

```
7
7
```

In [74]:
```
c==d
```

Out[74]: True

In [75]:
```
c!=d
```

Out[75]:
```
False
```

In [76]:
```
print(c)
print(d)
```

```
7
7
```

In [77]:
```
c>d
```

Out[77]:
```
False
```

In [78]:
```
c>=d
```

Out[78]:
```
True
```

In [79]:
```
c<d
```

Out[79]:
```
False
```

In [80]:
```
c<=d
```

Out[80]:
```
True
```

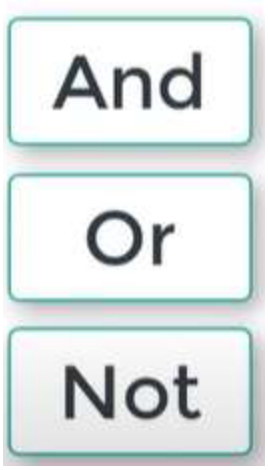
In [ ]:
```

```

Logicl operators

you need to understand about true and false table

3 important part of logic operator is--- AND,OR,NOT



In [81]:
```
a=5
b=4
```

In [82]:
```
a<8 and b<5
```
Out[82]: True

In [49]:
```
a<8 and b<2
```
Out[49]: False

In [50]:
```
a>8 or b<2
```
Out[50]: False

In [83]:
```
a<8 or b<5
```
Out[83]: True

In [84]:
```
print(a)
print(b)
```
```
5
4
```

In [85]:
```
a<8 and b<2
```
Out[85]: False

In [86]:
```
a>8 or b<2
```
Out[86]: False

In [88]:
```
x=True
x
```
Out[88]: True

In [51]:
```
x= False
x
```
Out[51]: False

In [56]:
```
not x # you can reverse the operation
```
Out[56]: True

In [59]:
```
x = not x
x
```
Out[59]: True

In [60]:
```
x
```
Out[60]: True

In [61]:
```
not x
```
Out[61]:
```
False
```
In [53]:
```
-True
```
Out[53]:
```
-1
```
In [54]:
```
-False
```
Out[54]:
```
0
```
In [ ]:
```
```